09-10: introduction

Administrative notes

You'll have weekly lectures and weekly practice classes (both mandatory to attend). Make sure you are registered for a practice class as you need to complete that before taking the theory exam. Practice requirements and course organisation are the practice tutor's responsibility.

Lectures are scheduled for Thursdays at noon and will start with little delay, so please make sure to arrive to the lecture on time.

Lecture material will be published in Canvas Pages, important notices will be given via Neptun course messages.

Prerequisites

The Compilers subject heavily builds on material covered in FOTOC (Fundamentals of theory of computation), such as formal grammar and automata theory. Therefore, we strongly advise you to complete that subject before taking the Compilers.

In case you decide to take Compilers before or with FOTOC, note that you need to self-study that subject since the Compilers instructors will assume from the first week that you are familiar with formal grammars, formal languages, and automata theory.

Requirements

Lecture course grading will be based on the exams taking place in the examination weeks. Continuous assessment is not applied, but short, non-graded quizzes may be offered at times.

Besides the exams, you can apply to make a short (15-min) presentation about a narrowed down topic of your interest. Note that such a presentation bust be based on established, recognized sources of information, such as books and academic papers, with exact citation. Should you decide to volunteer to make a presentation, you have to agree on the topic with the lecturer by the end of the 6th week of teaching. Student presentations can be given in the presentations' event (either in the last teaching week or in the first exam week, to be decided) or in exceptional cases can be recorded and distributed among all course participants.

Introduction to the topic

Why do we need compilers?

Computers understand simple instructions, written in the so-called machine code language. You can instruct a computer to do simple arithmetic operations, jump from one instruction to another, but high-level constructs (such as variables, types, advanced control statements) are not available. Machine code is inherently low-level, verbose, difficult to read and write. In contrast, problems we want to solve with

the computer tend to be higher-level, simpler to be expressed with formulas and algorithms. There is a significant abstraction gap between these two, which is bridged by compilers. The compiler is essentially a translator: it understands high-level languages like C, and translates them into low-level expression of the instructions.

There is another important caveat to machine code: it is specific to particular hardware and OS. However, the programs we write should be executable on the various machines and devices we use, without any reworking. Fortunately, compilers allow us to express programs in a target platform-independent way, and they take care of the specialization automatically. Will the translated low-level code be as good as a hand-optimised one? Probably not, but modern compilers do heavy and smart optimisations so that they can achieve excellent performance (in terms of space and time), comparable to hand-optimised versions.

An example

Take as example the program finding the maximal value in an integer array (of length 5, containing the values 1,40,5,-60 and 7). In C, you would express it like this:

```
#include <stdio.h>

int len = 5;
int arr[] = {1,40,5,-60,7};

void main()
{
  if(len > 0) {
    int max = arr[0];
    for(int i = 1; i < len; i++)
      if(arr[i] > max) max = arr[i];
    printf("%d\n", max);
  }
}
```

In contrast, the same (well, fairly similar) algorithm is expressed in assembly (let's say, low-level code) in the following way:

```
global main
extern write_int

section .data
len: dd 5
arr: dd 1,40,5,-60,7

section .bss
max: resd 1

section .text
main:
  cmp dword [rel len], 1
  jb error
```

```
  mov eax, [rel arr]
  mov [rel max], eax
  mov ecx, 1
loop:
  cmp ecx, [rel len]
  jae end
  lea eax, [arr+ecx*4]
  mov eax, [eax]
  inc ecx
  cmp eax, [rel max]
  jg setmax
  jmp loop
setmax:
  mov [rel max], eax
  jmp loop
end:
  push dword [rel max]
  call write_int
  add esp, 4
error:
  ret
```

It is obvious to see how different the abstraction level is and how more verbose the latter one is (in the lecture we discussed some concrete instructions in the above snippets, but we will explain the differences in detail in the course of the course. By the end of the semester, you will be able to read and write assembly code like this, and what's more important, you will be able to generate it from a code similar to the one above.

Language implementations: compilers and interpreters

It is common to identify two main kinds of programming language implementations (some languages use a mix thereof): compilers and interpreters. A compiler emits code that is specific to the target; if you want to execute a program on a different target, you need a compiler for that platform, and recompile the source. In contrast, interpreters execute the source program directly, without specialising the program to the target, although they themselves are specific to the target architecture. If one only compares execution times (without considering compilation time), interpreters tend to be slower; however, modern interpreters do use just-in-time compilation to optimise the most frequently executed instructions to the target machine.

There is another important observation to make: the architecture of the compiler seems to be reused not only in interpreters, but also in language servers, static analysers or refactoring tools. Indeed, "understanding" the source code is among the main tasks of all these tools, so it is not surprising that they perform similar analysis steps to build an intermediate representation from the program source code.

Note: in the lecture we also discussed the concepts of language specification and undefined behaviour, and how object code behaviour may very among the different compilers for the same language.

A bit of history

High-level programming languages started to evolve around the sixties. It turned out that there are different formalisms from which one can synthesise computer code, or put differently, there are different approaches in which one can abstract away from the machine level. One approach is called imperative: various methods to instruct the machine to maintain and manipulate a global state, via sophisticated control statements and error handling. A different approach is called declarative: declare the computation to be done via pure, mathematical formulas, and let the computer evaluate the formulas following a particular strategy. A programming paradigm stemming from the latter is called functional, and many modern software systems now rely on functional programming languages.

Describing tasks in higher-level languages precedes compilers. In fact, the first compilers were humans, who understood the program described in more abstract terms (but a semiformal language) and expressed the same program on the level of the machine. With the many advances in handling formal languages and computation models, the theory of compilers has evolved into what it is today.

The anatomy of a compiler

Almost all compilers are divided into two groups of steps: analysis (frontend) and synthesis (backend), connected via a so-called intermediate representation (IR). The aim of analysis is to understand the source program and capture structural and semantic information in the IR; then comes synthesis to do code generation and optimisation.

Analysis takes the input text, and does the following crucial steps:

- Lexical analysis: chop the text (list of characters) into words/symbols (also called tokens). Intuitively, what this means is separating the text based on white spaces and punctuations (like one would do in a natural language text), but it will turn out that the task here is more difficult.
- Syntax analysis: the sequence of lexical symbols is checked for structural integrity, whether the words in the text follow the grammatical rules prescribed in the syntax of the language. This task is complex in itself, but the most interesting aspect is finding an algorithm with a good performance (time and space complexity).
- Semantic analysis: the structure tree recovered in the syntax analysis is then checked for the so-called context-dependent properties, the rules of the language that cannot be encoded in context-free grammar rules. These typically include variable bindings and type checking.

Why is the analysis done in these various steps rather than in a single step? Because decomposing the problem to subproblems typically leads to simpler and more effective solutions, better error reporting and error handling, and it is, in general, easier to reason about. Interestingly enough, the three steps correspond to the Chomsky 3, 2 and 1 language classes. We will study how the various phases are defined formally, and we obtain efficient algorithms to decide if a program text really belongs to the source language's rules.

Then, code synthesis takes the analysed program and translates it. Synthesis is only partly covered in our lecture, we will focus on code generation, and less on optimisation. Code generation will be done via code generation templates, which lower the abstraction level of the code and express it in the target language at the same time.

See **https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/handouts/020%20CS143%20Course%20Overview.pdf (https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/handouts/020%20CS143%20Course%20Overview.pdf)** for more details.